

DIGITAL FORENSICS LABORATORY · LAB 06

Disk Image Forensics

Working paper — imaging, FTK Imager, \$MFT records, deleted files, and honest timestamps

Name	Student ID (NIM)	Class / Group
------	------------------	---------------

Date	Instructor / TA	Session grade
------	-----------------	---------------

1. Lab Overview

This working paper accompanies Lab 06 of the digital forensics series. The disk is the deepest witness yet: it keeps files you can see, files someone tried to delete, and the filesystem's own bookkeeping - timestamps, parent directories, hidden streams. You will acquire and open forensic images with FTK Imager, interrogate NTFS metadata files like \$MFT and \$Boot, recover deleted content, and learn to catch forged timestamps by comparing the two clocks NTFS keeps. Complete the pre-lab before the session, check off each step as you perform it, cite record numbers and offsets in every answer, and close with a reflection.

1.1 Learning objectives

- Explain disk image, disk imaging, and image forensics - and the hash discipline that ties them together.
- Acquire an image with FTK Imager and open existing images (Raw/dd, AD1, E01).
- Navigate FTK's four panes and export files (and alternate data streams) as evidence.
- Name the NTFS metadata files (\$MFT, \$MFTMirr, \$LogFile, \$Boot) and their forensic value.
- Parse \$MFT records: record numbers, \$FILE_NAME parent refs, attribute types, resident data.
- Use MFTECmd / analyzeMFT to turn \$MFT and \$Boot into readable CSV.
- Recover deleted files from FAT and NTFS images, and justify why the bytes survived.
- Detect timestamping by comparing \$STANDARD_INFORMATION against \$FILE_NAME times.

1.2 Environment and files

Environment: Windows with FTK Imager installed (free download from Exterro) and MFTECmd (Eric Zimmerman). Copy every image to your working directory first and hash it - images are evidence, and exports from them are evidence too.

Table 1.1 — Lab files

File	Purpose
files/Image.ad1	Challenge A — FTK logical image of the suspect USB drive (upstream)
files/suspect_card.dd	Challenge B (new) — raw FAT16 image of a shipping office's USB stick
files/mini_mft.bin	Challenge C (new) — sixteen crafted \$MFT records to dissect

1.3 What to submit

- Section 1.4 pre-lab, answered before the session (worth 10 points).
- This working paper as PDF, named NIM_Name_Lab06.pdf.
- Your evidence folder NIM_Lab06/evidence/ containing every numbered exhibit (B-01, B-02, and so on).
- Answers that cite exact record numbers, clusters, offsets, or hashes - not vague descriptions.

1.4 Pre-Lab Check

Answer these before the session, using the lab README and Section 2 of this paper as references. One or two sentences per question are enough - precision beats length.

P1. What is the difference between copying files from a disk and imaging the disk - name two things an image preserves that a file copy does not.

Answer:

Two sentences maximum.

P2. Why do we hash an image at acquisition time AND again after analysis? What does each hash prove?

Answer:

Two sentences maximum.

P3. What is the \$MFT, and what four things does each record hold that interest an investigator?

Answer:

Two sentences maximum.

P4. In NTFS, which attribute stores the timestamps shown in Explorer, and why is the other timestamp attribute harder to fake?

Answer:

Two sentences maximum.

P5. What happens to a file's directory entry and data when it is deleted on FAT - and why is the content often still recoverable?

Answer:

Two sentences maximum.

2. Theory Primer

2.1 Images and the hash discipline

A disk image is a bit-by-bit copy of a device or partition - visible files, deleted content, slack space, and filesystem metadata included. Imaging tools write in several formats: Raw (dd) stores every byte with no frills; AD1 is FTK Imager's logical format; E01 adds compression and embedded hashes. Whatever the container, the discipline is constant: hash at acquisition, analyze copies only, hash after analysis. Matching hashes are your proof that the original was never touched - the whole game of admissibility in one sentence.

2.2 NTFS metadata files

Table 2.1 — The files the filesystem keeps for itself

File	Holds	Forensic value
\$MFT	one 1024-byte record per file and folder	names, parents, times, sizes, deleted records
\$MFTMirr	copy of the first \$MFT records	repair and redundancy
\$LogFile	journal of metadata changes	what changed on the volume, and when
\$Boot	volume boot sector	serial number, geometry - the volume's ID card
\$Extend / \$RmMetadata	extension objects, transactional data	where \$Txf and friends actually live

2.3 Reading an \$MFT record

Each record starts with the FILE magic; its number in the table is the file's identity (record 40 is note.txt in this lab's image). Inside, attributes do the work: 0x10 \$STANDARD_INFORMATION carries the timestamps user tools display; 0x30 \$FILE_NAME carries the name, the parent

directory's record reference, and a second, kernel-written set of times; 0x80 \$DATA is the content - resident (stored inside the record itself) when small. Parent references chain records into the directory tree without reading a single directory listing.

2.4 Deletion, FAT vs NTFS

On FAT (USB sticks), deletion rewrites the directory entry's first byte to 0xE5 and clears the FAT chain - but the entry text and the data clusters survive until reuse. On NTFS, the MFT record is marked not-in-use while its content pointers usually remain intact. Either way: deleted means forgotten, not erased - and the recovery recipe is to find the forgotten pointer (or carve the bytes) and verify with the reported size.

2.5 Two clocks and timestomping

Attackers backdate files with SetFileTime-style tools, which rewrite only \$STANDARD_INFORMATION. The kernel-written \$FILE_NAME times (stored in the parent directory's record) are far harder to alter. The examiner's move: put both clocks side by side for every interesting file. Agreement is boring; wild disagreement - a file 'created in 2019' whose kernel times say 2026 - is timestomping until proven otherwise.

3. Guided Exercises

Work through the drills on files/Image.ad1. Every export is an exhibit: capture the FTK view, number it (B-01, B-02, and so on), and log it in Appendix B with its hash.

3.A Open and orient

- 1. Copy Image.ad1 to your working directory; record its MD5 and size.
- 2. FTK Imager > File > Add Evidence Item > Image File; open the copy.
- 3. Expand the evidence tree to the root of the imaged drive; list what you see (count files vs system files).
- 4. Select note.txt; record the timestamps AND the hashes from the Properties pane.

3.B Export and hash

- 1. Export note.txt (right-click > Export Files); verify its MD5 matches the Properties pane.
- 2. Export meme.jpeg and its Zone.Identifier stream (expand the file in the tree to find the ADS).
- 3. Record the HostUrl found inside the exported stream.
- 4. Hash every export and log all three hashes.

3.C Meet the \$MFT

- 1. Export the \$MFT file itself; record its size (hint: records = size / 1024).
- 2. Run MFTECmd: MFTECmd.exe -f \$MFT --csv . --csvf mft.csv.
- 3. Open the CSV: find note.txt's record number and its parent's record number.
- 4. Find record 31 (\$Txf): what is its parent, and what is THAT record's name?

3.D Boot sector basics

- 1. Export \$Boot; open it in FTK's hex view (or a hex editor).
- 2. Confirm bytes/sector (offset 0x0B) and sectors/cluster (0x0D).
- 3. Read the 8 bytes at offset 0x48 - the volume serial number; write it in hex.
- 4. Run MFTECmd on \$Boot and compare its parsed serial with your hex reading.

3.E Optional: image your own USB

- 1. Use a disposable USB stick: File > Create Disk Image > Physical Drive > Raw (dd).
- 2. Record the acquisition hash FTK reports at the end.
- 3. Mount your own image and find a file you know - verify its hashes match.
- 4. Store or securely wipe the image afterward; never image drives casually.

4. Challenges

Challenge A is the upstream USB image - solved together in the Deep Dive session. Challenges B and C are yours: work on copies, hash before and after, cite records, clusters, and offsets. Challenge A has no flag - its five answers are the artifacts. Challenges B and C use ctf{xxxx}.

4.1 Challenge A — the USB image (solved in the Deep Dive)

The five questions from the lab README: the MD5 and SHA1 of note.txt, its \$MFT record number, the parent directory of \$Txf, the original Twitter URL of meme.jpeg, and the raw-hex volume serial from \$Boot. The Deep Dive walks the full solution - your deliverable is the same answers backed by YOUR exhibits.

4.2 Challenge B — the shipping office's stick (files/suspect_card.dd)

- 1. Copy the image to your working directory and record its MD5 and exact size in bytes.
- 2. Open it in FTK Imager as a Raw image. What filesystem, volume label, and total size does FTK report?
- 3. List every entry in the root directory, including any that appear damaged or deleted. Quote the first byte of the deleted entry in hex.

- 4. What two fields of the deleted entry were changed or cleared on deletion - and which field still tells you the content's exact length?
- 5. Locate the deleted content in the data area (start at cluster 2 and walk forward; small deleted files stay contiguous). Record the cluster and byte offset.
- 6. Recover the content (carve it from hex view or export if your tool allows) and hash your recovered file.
- 7. Write down what the recovered plan says - drop point, signal, and the auth token (the flag).

Hints: (1) FTK's Properties pane shows the image's filesystem; the hex view of the root folder (sector 19 onward) shows the 0xE5. (2) The deleted entry's size field survived - that is how you know where the content ends. (3) Data clusters start right after the root folder area; the deleted file is the third file ever written to the stick.

Flag B: ctf{_____}

4.3 Challenge C — the sixteen records (files/mini_mft.bin)

- 1. Copy the file and record its MD5. It contains sixteen 1024-byte \$MFT-style records - which record numbers are in use (FILE magic present)?
- 2. For every in-use record, extract: flags (file or directory), \$FILE_NAME name, and parent record number.
- 3. Rebuild the directory tree on paper: which records live in the root, and which live inside the subdirectory?
- 4. For every file record, list the \$STANDARD_INFORMATION year and the \$FILE_NAME year side by side.
- 5. Exactly one file disagrees with itself. Name its record number, its path, and both claimed years.
- 6. Read that record's resident \$DATA attribute and write out what it confesses - including the flag.
- 7. One root-level file has a name that sounds suspicious but honest times. Which is it, and why do its clocks clear it?

Hints: (1) Record number = file offset / 1024; MFTECmd or analyzeMFT will parse it, or use a hex editor with the layout card from Section 2.3. (2) The disagreement to hunt is YEARS apart, not seconds. (3) Resident data sits inside the record, right after the attribute header.

Flag C: ctf{_____}

5. Analysis Questions

Questions 1-3 condense the original lab's five questions; questions 4-12 extend the case. Cite record numbers, offsets, clusters, or hashes in every answer.

1. The USB image (files/Image.ad1):

- a.** What are the MD5 and SHA1 of note.txt?
- b.** What is note.txt's \$MFT record number, and which record is its parent?
- c.** Which directory contains \$Txf - give the full chain up to the root.

Answer:

Write your complete answer, citing the exact record number, cluster, offset, or hash used.

Supporting evidence (screenshot / output): exhibit numbers B-_____ to B-_____

2. Origins and identity:

- a.** What original URL was meme.jpeg downloaded from, and in which artifact was it preserved?
- b.** What is the volume serial number of the imaged drive, in raw hex?

Answer:

Write your complete answer, citing the exact record number, cluster, offset, or hash used.

Supporting evidence (screenshot / output): exhibit numbers B-_____ to B-_____

3. Challenge B outcomes:

- a.** Which byte marks a deleted FAT entry, and which two bookkeeping structures did deletion actually touch?
- b.** Why did the deleted content survive at a predictable cluster despite the cleared FAT chain?

Answer:

Write your complete answer, citing the exact record number, cluster, offset, or hash used.

Supporting evidence (screenshot / output): exhibit numbers B-_____ to B-_____

4. Explain the difference between a logical image (AD1) and a physical image (raw dd) - and name one artifact each can miss that the other captures.

Answer:

Write your complete answer, citing the exact record number, cluster, offset, or hash used.

Supporting evidence (screenshot / output): exhibit numbers B-____ to B-____

5. Why does \$MFTMirr exist, and what exactly does it mirror? Could note.txt's record be recovered from it in this image?

Answer:

Write your complete answer, citing the exact record number, cluster, offset, or hash used.

Supporting evidence (screenshot / output): exhibit numbers B-____ to B-____

6. An examiner reports 'the file was created in 2019' from Explorer-style timestamps alone. Give the two-clock argument against trusting that, and name the exact attributes involved.

Answer:

Write your complete answer, citing the exact record number, cluster, offset, or hash used.

Supporting evidence (screenshot / output): exhibit numbers B-____ to B-____

7. A suspect claims a file was never on the USB stick. Which three artifacts from this lab could contradict that claim even though the file no longer appears in any directory?

Answer:

Write your complete answer, citing the exact record number, cluster, offset, or hash used.

Supporting evidence (screenshot / output): exhibit numbers B-____ to B-____

8. The \$LogFile records metadata transactions. Why is it paired with \$MFT in a serious investigation, even when \$MFT already parses cleanly?

Answer:

Write your complete answer, citing the exact record number, cluster, offset, or hash used.

Supporting evidence (screenshot / output): exhibit numbers B-____ to B-____

9. In Challenge B, the meeting notes reference 'the other file'. Explain how a recovered deleted file plus a live file together build a stronger case than either alone.

Answer:

Write your complete answer, citing the exact record number, cluster, offset, or hash used.

Supporting evidence (screenshot / output): exhibit numbers B-____ to B-____

10. Zone.Identifier exists only on NTFS. Where would the equivalent download-origin evidence live on a FAT-formatted stick, if anywhere? Justify your answer.

Answer:

Write your complete answer, citing the exact record number, cluster, offset, or hash used.

Supporting evidence (screenshot / output): exhibit numbers B-____ to B-____

11. Why must every exported file be hashed at export time? Connect this to the chain-of-custody concepts from Lab 01.

Answer:

Write your complete answer, citing the exact record number, cluster, offset, or hash used.

Supporting evidence (screenshot / output): exhibit numbers B-____ to B-____

12. Management asks: 'can we prove which computer this stick belonged to?' Using only this lab's artifacts, what CAN you honestly say - and what would you need to say more?

Answer:

Write your complete answer, citing the exact record number, cluster, offset, or hash used.

Supporting evidence (screenshot / output): exhibit numbers B-____ to B-____

6. Reflection and Self-Assessment

Summarize your main findings, the skills that clicked, and the obstacles you hit - with how you solved them.

Reflection:

Findings, new skills, obstacles, and how you overcame them.

Supporting evidence (screenshot / output): exhibit numbers B-____ to B-____

Table 6.1 — Self-assessment checklist

I can	Yes	Not yet
Acquire an image with FTK Imager and verify its hash		
Navigate FTK's four panes and export files with hashing		
Name the NTFS metadata files and their value		
Parse \$MFT records: numbers, parents, attributes		
Run MFTECmd on \$MFT and \$Boot and read the CSV		
Recover a deleted file from a FAT image by hand		
Detect timestomping via SI vs FN comparison		

7. Appendices

Appendix A — Tool cheat sheet

Task	Command / action
Open an image	FTK: File > Add Evidence Item > Image File

Task	Command / action
Acquire raw image	FTK: File > Create Disk Image > Physical Drive > Raw (dd)
Export a file / ADS	right-click > Export Files (hash it immediately)
Parse \$MFT to CSV	MFTECmd.exe -f "\$MFT" --csv . --csvf mft.csv
Parse \$Boot	MFTECmd.exe -f "\$Boot" --csv . --csvf boot.csv
Python MFT parser	python analyzeMFT.py -f \$MFT -o records.csv
Filter the CSV	Import-Csv mft.csv ? Name -eq 'note.txt'
Record number from hex	offset of FILE magic / 1024
FAT deleted marker	0xE5 in the first byte of the 32-byte directory entry
\$Boot serial offset	0x48, 8 bytes, report as hex string
Hash an export	Get-FileHash -Algorithm MD5 recovered.bin

Appendix B — Evidence log

One row per exhibit. Keep numbering continuous across the whole session.

What good evidence looks like — model exhibit:

```
[B-03] 2026-10-26 09:47   FTK Imager, Image.ad1
$MFT record 40 (offset 0xA000): FILE0, flags 0x0001
  $FILE_NAME: note.txt, parent ref 5 (root)
-> captured as evidence/B-03_mft_rec40.png
```

- The record number, offset, and attribute values are fully visible.
- The tool and moment of capture are recorded; the filename says what it shows.
- One exhibit supports one finding.
- Bad evidence: 'I found the record in MFTECmd' with no record number or screenshot.

No.	Time	Activity / Tool	Source file	Notes
B-01				
B-02				
B-03				
B-04				
B-05				
B-06				
B-07				

No.	Time	Activity / Tool	Source file	Notes
B-08				
B-09				
B-10				

Appendix C — Hash verification form

Fill one row per image and export: hash when received/created (start) and after analysis (end). Identical values prove faithful handling.

No	Image / export	Algorithm	Hash (start)	Hash (end)	Match?
1					Yes / No
2					Yes / No
3					Yes / No
4					Yes / No
5					Yes / No
6					Yes / No

Appendix D — Glossary

Term	Meaning in this lab
disk image	bit-by-bit copy of a device: files, deleted data, slack, metadata
raw (dd) / AD1 / E01	the common image formats: plain bytes / FTK logical / EnCase compressed
\$MFT	master file table: one 1024-byte record per file/folder on NTFS
MFT record number	the file's identity - offset within \$MFT divided by 1024
attribute 0x10 / 0x30 / 0x80	\$STANDARD_INFORMATION / \$FILE_NAME / \$DATA
parent reference	the record number of the containing directory, stored in \$FILE_NAME
resident data	file content small enough to live inside the MFT record itself
\$MFTMirr / \$LogFile	mirror of early MFT records / metadata transaction journal
\$Boot	boot sector: geometry plus the volume serial number (offset 0x48)
alternate data stream (ADS)	named NTFS stream - e.g. Zone.Identifier with the download URL
0xE5	FAT's deleted-entry marker in the first byte of a directory entry

Term	Meaning in this lab
timestomping	backdating a file by rewriting \$STANDARD_INFORMATION times
SI vs FN (two clocks)	user-writable times vs kernel-written \$FILE_NAME times
carving	recovering content by scanning raw bytes when pointers are gone
unallocated space	clusters not assigned to any live file - where deleted data waits
chain of custody	the hash-verified handling trail from seizure to report

Honesty statement: by submitting this working paper I declare that all answers and evidence are my own work, that original evidence files were never modified, and that I followed the lab's ethics rules.

Student signature: _____ Date: _____

Instructor / TA signature: _____ Grade: _____